CS 360
Programming Languages
Day 22

Review

- Syntax/Semantics
- Grammar
- bf: 30,000 bytes and a pointer, instructions for manipulating bytes, the pointer, I/O, and looping

Warmup: Funstacker

- Use the following grammar to produce some math-expressions:

```
m-expr  : m-list | integer
m-list  : "(" func (" "+ m-expr)* ")"
integer : ["-"] digit+
digit   : "0" | "1" | "2" | "3" | "4"
        | "5" | "6" | "7" | "8" | "9"
func    : "+" | "*"
```

Try `(* 2 (+ 3 4) 5)`

Try `(+ (* -2 3 4) 1)`

bf

- Produce the following programs with the following grammar:

```
>>>,++.
```

```
+[>+]>.
```

```
bf-program : (bf-op | bf-loop)*
bf-op      : ">" | "<" | "+" | "-" | "." | ","
bf-loop    : "[" bf-program "]"
```

Racket Grammar

Exercise:

- Make a grammar that produces a valid assignment in Python. Be as complete as possible!

- Examples:

```
my_var = my_val # valid!
1my_var = my_val # invalid!
my_var = my_val + # invalid!
```

Parsing

- **Parsing** is the act of figuring out which grammar rules produce a certain string

- **Top-down** vs **bottom-up** parsing: how do you process the text to find a valid parse tree?

Parsing in Racket

- Wouldn’t it be nice if we could just tell Racket our intended grammar, and it does all the parsing for us?

brag

a Better Racket AST Generator

(“abstract syntax tree”)

aka a language for parsing!

“Sentence” Grammar

- Let’s make a simple grammar which produces “English” “sentences”:

- Subject Verb [Object]

“Assignment” Grammar

- Let’s make a grammar that produces simple assignment statements:

```
var = val o val
# var is a valid variable name
# val is a variable or a number
# o is a valid operation
```

- What is “fragile” about our grammar?

bf grammar

```
bf-program : (bf-op | bf-loop)*
bf-op      : ">" | "<" | "+" | "-" | "." | ","
bf-loop    : "[" bf-program "]"
```

- Parse a few programs:

```
(parse-to-datum program-string)
```

Problems with Parsers

- By default, parsers handle input character by character. What's wrong with this?

- How could we fix this?

Lexers / Tokenizers

- A “token” or “lexeme” is the smallest meaningful unit in a language
- It would be great if we could pre-process the source code before attempting to parse it
- At this stage of processing, we could filter out whitespace and comments
- And we could even detect illegal tokens before attempting to parse!
- More on Monday!